Mini Project Report

on

UML Class Diagram Code Generator in C++

Submitted by

Project Members

Roll No	PRN No	Name
7	1262241984	Mahek Shukla
10	1262241998	Rohan Upadhye
12	1262242010	Chirayu Agrawal
20	1262242058	Yatharth Agrawal

SY.CSE (Panel E)

Course : Object Oriented Programming in CPP

Course Code : CSE1003B

Under the Internal Guidance of

Prof. Shraddha Oza

Department of Computer Engineering and Technology

MIT World Peace University, Kothrud,

Pune 411 038, Maharashtra - India

2025 - 2026

DEPARTMENT OF COMPUTER ENGINEERING AND TECHNOLOGY

C E R T I F I C A T E

This is to certify that,

Mahek Shukla, Rohan Upadhye, Chirayu Agrawal, Yatharth Agrawal

of **SY - BTech. (Computer Science & Engineering)** have completed their mini project titled ***"UML Class Diagram Code Generator in C++"*** and have submitted this Object Oriented Programming in CPP Report for the academic year 2025 - 26 (Sem - IV).

Prof. Shraddha Oza
Mini Project Guide
Department of CET
MIT World Peace University, Pune

Abstract

This project implements a UML Class Diagram Code Generator in C++ that automates the conversion of structured text - based UML descriptions into compilable C++ header (.h) files. The system reads a plain - text input file (diagram.txt) containing class definitions with attributes and methods, parses them into an in - memory object model, and generates properly formatted C++ code with correct access specifiers (public, private, protected) and inheritance relationships.

The project is built entirely using Object Oriented Programming principles including Encapsulation, Abstraction, Inheritance, Polymorphism, Function Overloading, Operator Overloading, Copy Constructors, and Virtual Destructors. The architecture follows a clean pipeline: Input Parser → UML Model Classes → Code Generator → File Writer, making it extensible for future language targets such as Java or Python.

Chapter 1 - Introduction

The UML (Unified Modeling Language) Class Diagram is a standard tool used by software engineers to design and document the structure of a system before writing any code. It shows classes, their attributes, their methods, and the relationships between them. However, translating a UML diagram into actual source code is a tedious and repetitive task.

This project addresses that problem by building an automated UML - to - C++ code generator. Given a simple text description of a UML diagram, the program automatically produces ready - to - compile C++ header files. This eliminates manual transcription errors and significantly speeds up the prototyping process.

The generator is implemented entirely in C++ and demonstrates the full spectrum of Object Oriented Programming concepts as required by the CSE1003B curriculum. The system is designed to be modular and extensible - adding support for a new output language (e.g., Java) requires only creating a new subclass of the abstract CodeGenerator base class.

Chapter 2 - Problem Statement

Software developers often begin system design using UML Class Diagrams. However, manually converting these diagrams into C++ class declarations is repetitive, error - prone, and time - consuming - especially when dealing with multiple classes, inheritance hierarchies, and access specifier rules.

There is a need for a tool that can:

- Accept a structured text description of a UML Class Diagram as input
- Automatically parse class names, attributes, methods, and relationships
- Generate compilable C++ header files with correct syntax and access specifiers
- Handle inheritance relationships (EXTENDS keyword)
- Be extensible to support future output languages

Chapter 3 - Objectives

1. Design and implement a text - based UML parser in C++ using file I/O and STL
2. Build an in - memory OOP model using UMLAttribute, UMLMethod, and UMLClass
3. Implement an abstract CodeGenerator base class demonstrating Abstraction and Polymorphism
4. Create a CPPCodeGenerator subclass that inherits from CodeGenerator and generates valid C++ header files
5. Demonstrate all 8 OOP concepts: Encapsulation, Abstraction, Inheritance, Polymorphism, Function Overloading, Operator Overloading, Copy Constructor, and Virtual Destructor
6. Support inheritance relationships in the generated code (class A : public B)
7. Produce a working, compilable, end - to - end system

Chapter 4 - List of Classes and Architecture

4.1 Class List

Class	File	Purpose
`UMLAttribute`	UMLAttribute.h	Stores one field: access + type + name. Has copy constructor.
`UMLMethod`	UMLMethod.h	Stores one method: access + returnType + name. Overloaded constructors.
`UMLClass`	UMLClass.h	Main container. Has operator+ and addAttribute/addMethod overloads.
`UMLParser`	UMLParser.h	Reads diagram.txt line by line. Builds vector<UMLClass>.
`CodeGenerator`	CodeGenerator.h	Abstract base class with pure virtual generate(). Virtual destructor.
`CPPCodeGenerator`	CPPCodeGenerator.h	Inherits CodeGenerator. Overrides generate() to write .h files.

4.2 System Architecture

The system follows a linear pipeline architecture:

```
diagram.txt
     |
     v
[ UMLParser ]          → reads lines, tokenizes, builds UMLClass objects
     |
     v
[ UML Model Classes ]  → UMLAttribute, UMLMethod, UMLClass in memory
     |
     v
[ CodeGenerator ]      → abstract base class (pure virtual generate())
     |
     v
[ CPPCodeGenerator ]   → overrides generate(), writes .h files
     |
     v
output/Student.h, Teacher.h, Person.h, Course.h ...
```

Chapter 5 - Description of OOP Concepts Used

5.1 Encapsulation

Encapsulation is implemented in all three model classes. UMLAttribute bundles its three data fields (access, type, name) with a constructor and display() method. UMLMethod bundles access, returnType, name, and a parameter vector. UMLClass bundles className with vectors of attributes and methods. In the generated C++ code, private data is kept separate from public methods - enforcing encapsulation in the output as well.

5.2 Abstraction

The CodeGenerator class is a pure abstract class with a pure virtual generate() function. It defines WHAT a code generator must do without specifying HOW. The caller only interacts with the abstract interface - it does not need to know which language is being generated. The UMLParser also demonstrates abstraction by hiding all parsing logic behind a simple parseFile() method.

5.3 Inheritance

CPPCodeGenerator inherits publicly from CodeGenerator. It inherits the printStatus() method and is required by the compiler to implement generate(). In the output files, inheritance is also generated: if the input diagram.txt contains CLASS Student EXTENDS Person, the generator outputs class Student : public Person { ... }.

5.4 Polymorphism

In main.cpp, a base class pointer (CodeGenerator* gen) is assigned a CPPCodeGenerator object. When gen - >generate(cls) is called, C++ uses virtual dispatch to call CPPCodeGenerator::generate() at runtime. Both generate() and printStatus() are virtual and overridden in the subclass. This means a future JavaCodeGenerator could be plugged in with zero changes to main.cpp.

5.5 Function Overloading

UMLClass provides two versions of addAttribute() and two versions of addMethod(). The first version accepts a full object (UMLAttribute or UMLMethod). The second version accepts raw strings and internally constructs the object. UMLMethod also provides two constructors - one without parameters and one with an explicit parameter list. The compiler selects the correct version based on the argument types.

5.6 Operator Overloading

UMLClass overloads the + operator twice - once for UMLAttribute and once for UMLMethod. This allows the intuitive syntax: student = student + UMLAttribute(...) and student = student + UMLMethod(...). The operator detects whether the right - hand side is an attribute or a method and routes to the appropriate internal vector.

5.7 Copy Constructor

Both UMLAttribute and UMLMethod define explicit copy constructors. These are invoked automatically when STL vectors resize (which requires copying existing elements to a new memory location) and when objects are passed by value to functions. The copy constructor ensures each string field and the params vector are properly deep - copied into the new object.

5.8 Virtual Destructor

CodeGenerator declares a virtual destructor. This is essential OOP best practice when using inheritance and base - class pointers. In main.cpp, delete gen is called on a CodeGenerator* pointer that actually holds a CPPCodeGenerator. Without a virtual destructor, only the base class destructor would run, causing a resource leak. With it, both CPPCodeGenerator::~CPPCodeGenerator() and CodeGenerator::~CodeGenerator() are called in order.

Chapter 6 - Implementation Details and Code

6.1 Input Format (diagram.txt)

The input file uses a simple keyword - based format designed for easy parsing:

```
// Comment lines are ignored
CLASS Student EXTENDS Person
ATTR  private  string  name
ATTR  private  int     rollNo
METHOD public  string  getName()
METHOD public  void    setName(string name)
END
```

6.2 UMLParser - parseFile()

The parser uses ifstream and getline to read the file line by line. Each line is tokenized using istringstream. Keywords CLASS, ATTR, METHOD, and END drive a simple state machine. When CLASS is encountered a new UMLClass is created. ATTR and METHOD tokens are parsed and added to the current class. END saves the completed class to the output vector.

6.3 CPPCodeGenerator - generate()

The generator iterates through the UMLClass object's attribute and method vectors. It writes the class declaration first (including : public ParentClass if applicable), then loops through attributes and methods, placing each in the correct access section. Include guards are added automatically based on the class name.

6.4 Sample Generated Output - Student.h

```
#ifndef STUDENT_H
#define STUDENT_H

#include <string>
#include <vector>
using namespace std;

class Student : public Person {
private:
    int rollNo;
    float marks;
    string branch;

public:
    int getRollNo();
    float getMarks();
    void setMarks(float marks);
    void display();
};

#endif
```

Chapter 7 - Test Cases and Results

#	Test Case	Input	Expected / Actual Output
1	Simple class with private attrs and public methods	CLASS StudentATTR private string nameMETHOD public string getName()END	output/Student.h with private string name; and public string getName();
2	Inheritance via EXTENDS keyword	CLASS Student EXTENDS Person	class Student : public Person { ... }
3	Protected access specifier	ATTR protected string name	Placed in protected: section in .h file
4	Multiple classes in one file	4 classes in diagram.txt	4 separate .h files generated
5	Comment lines ignored	// This is a comment	Line skipped, no parse error
6	Empty/blank lines ignored	Blank lines between CLASS blocks	Skipped cleanly, no crash
7	operator+ demo	extra = extra + UMLAttribute(...)	ExtraDemo.h generated correctly
8	Virtual destructor called	delete gen on CodeGenerator*	Both destructors print and run

Chapter 8 - Individual Contributions

Roll	Name	Contribution
7	Mahek Shukla	Designed the complete project architecture; implemented all core files, abstract CodeGenerator with pure virtual function, CPPCodeGenerator with inheritance and polymorphism, and virtual destructor.
10	Rohan Upadhye	Implemented UMLClass.h with operator+ overloads for both types and two-version function overloads for addAttribute() and addMethod().
12	Chirayu Agrawal	Implemented copy constructors and default constructors for UMLAttribute and UMLMethod; designed diagram.txt input format and wrote all test cases.
20	Yatharth Agrawal	Implemented UMLParser with ifstream, istringstream, tokenize() helper, and keyword-driven state machine for end-to-end file parsing.

Chapter 9 - Conclusion and Future Scope

9.1 Conclusion

This project successfully implements a fully functional UML Class Diagram Code Generator in C++. The system reads a structured text file, parses it into an in - memory object model, and automatically generates compilable C++ header files. All 8 required OOP concepts are demonstrated in working code - not just theoretically described.

The project validates the practical utility of OOP design: the abstract CodeGenerator interface makes the system extensible, operator overloading makes the API intuitive, the copy constructor ensures safe memory management inside STL containers, and the virtual destructor ensures correct cleanup in a polymorphic context.

9.2 Future Scope

- JavaCodeGenerator - subclass of CodeGenerator that generates Java .java files
- PythonCodeGenerator - generates Python class definitions
- GUI Interface - drag - and - drop UML diagram builder using Qt or SFML
- Support for associations, aggregations, and dependency relationships
- Auto - generation of getter/setter method bodies (not just declarations)
- Bidirectional conversion - parse existing C++ code into UML text

References

8. GeeksforGeeks - Object Oriented Programming in C++: https://www.geeksforgeeks.org/object - oriented - programming - in - cpp/
9. cplusplus.com Reference - Standard Library: https://cplusplus.com/reference/
10. Scribd - OOP Project Ideas: https://www.scribd.com/document/444916057/OOP - Project - Ideas - pdf
11. UML Class Diagram Notation - uml - diagrams.org: https://www.uml - diagrams.org/class - diagrams - overview.html
12. C++ File I/O - cppreference.com: https://en.cppreference.com/w/cpp/io/basic_ifstream